Formal Grammar

Concepts of Programming Languages

CAS CS 320

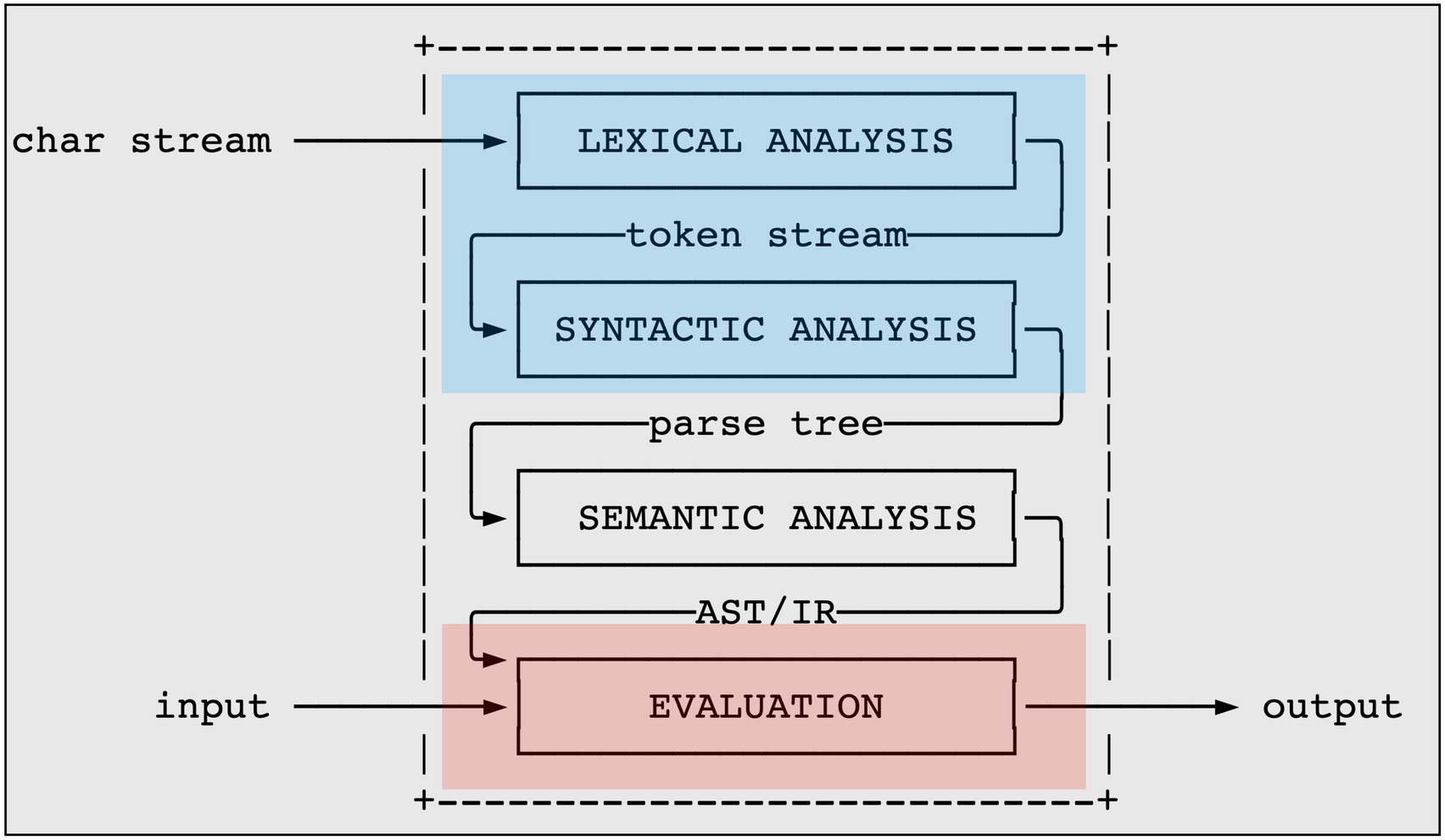
Outline

- Discuss briefly the **interpretation pipeline**
- Introduce **formal grammars,** a mathematical framework for syntax and parsing
- Discuss grammatical **ambiguity**

The Interpretation Pipeline

The Picture

parsing (this week)



semantics (after the midterm)

Today

We need a formal language for describing the syntax of programming languages

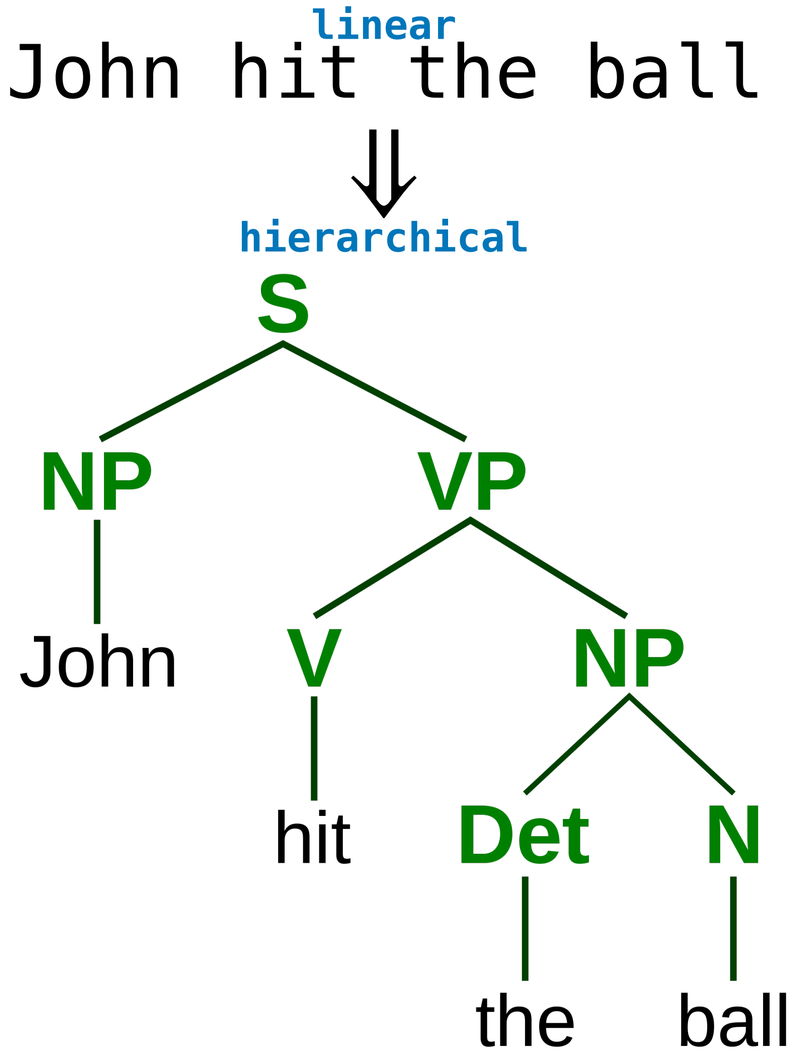
This is part of the study of **formal language theory**

Nearly every PL out there (including OCaml) is described using **Backus–Naur Form (BNF) Grammars**

What is Grammar?

Grammar refers to the rules which govern what statements are well-formed

Grammar gives **linear** statements (in natural language or code) their **hierarchical** structure



https://commons.wikimedia.org/wiki/File:ParseTree.svg

Grammar vs. Semantics

I taught my car in the refrigerator. 

vs.

My the car taught I refrigerator. 

Grammar is not about meaning, just structure

(As we will see, it is useful to separate these two concerns)

Grammars for Programming Languages

Formal grammars for PL tell us which programs are well-formed

Well-formed programs don't need to be meaningful

(In OCaml, well-formed programs are the ones we can type-check)

```
# let f x = x + 1;;
val f : int -> int = <fun>
# let rec x = x x x x ;;
Line 1, characters 14-15:
1 | let rec x = x x x x ;;
                  ^
Error: This expression has type ...
       but an expression was ex ...
       The type variable 'a occ ...
# let rec f x = f x + 1 - 1;;
val f : 'a -> int = <fun>
# let x = List.hd [];;
Exception: Failure "hd".
# let x = ;;
Line 1, characters 8-10:
1 | let x = ;;
            ^^
Error: Syntax error
```

How do we formally represent well-formed sentences?

An Example

the cow jumped over the moon

How do we know this a well-formed sentence?

An Example

```
<article> cow jumped over the moon
```

An Example

<article> <noun> jumped over the moon

An Example

<noun-phrase> jumped over the moon

An Example

<noun-phrase> jumped over <article> moon

An Example

<noun-phrase> jumped over <article> <noun>

An Example

<noun-phrase> jumped over <noun-phrase>

a thing jumped over a thing

An Example

<noun-phrase> jumped <prep> <noun-phrase>

An Example

<noun-phrase> jumped <prep-phrase>

An Example

<noun-phrase> <verb> <prep-phrase>

An Example

<noun-phrase> <verb-phrase>

a thing did a thing

An Example

<sentence>

We know it's a sentence because it has the right kind of hierarchical structure

A Derivation

<sentence>
<noun-phrase> <verb-phrase>
<noun-phrase> <verb> <prep-phrase>
<noun-phrase> jumped <prep-phrase>
<noun-phrase> jumped <prep> <noun-phrase>
<noun-phrase> jumped over <noun-phrase>
<noun-phrase> jumped over <article> <noun>
<noun-phrase> jumped over <article> moon
<noun-phrase> jumped over the moon
<article> <noun> jumped over the moon
<article> cow jumped over the moon
the cow jumped over the moon

A Derivation

<sentence>
<noun-phrase> <verb-phrase>
<noun-phrase> <verb> <prep-phrase>
<noun-phrase> jumped <prep-phrase>
<noun-phrase> jumped <prep> <noun-phrase>
<noun-phrase> jumped over <noun-phrase>
<noun-phrase> jumped over <article> <noun>
<noun-phrase> jumped over <article> moon
<noun-phrase> jumped over the moon
<article> <noun> jumped over the moon
<article> cow jumped over the moon

the cow jumped over the moon

A Derivation

<sentence>
<noun-phrase> <verb-phrase>
<noun-phrase> <verb> <prep-phrase>
<noun-phrase> jumped <prep-phrase>
<noun-phrase> jumped <prep> <noun-phrase>
<noun-phrase> jumped over <noun-phrase>
<noun-phrase> jumped over <article> <noun>
<noun-phrase> jumped over <article> moon
<noun-phrase> jumped over the moon
<article> <noun> jumped over the moon

<article>

the cow jumped over the moon

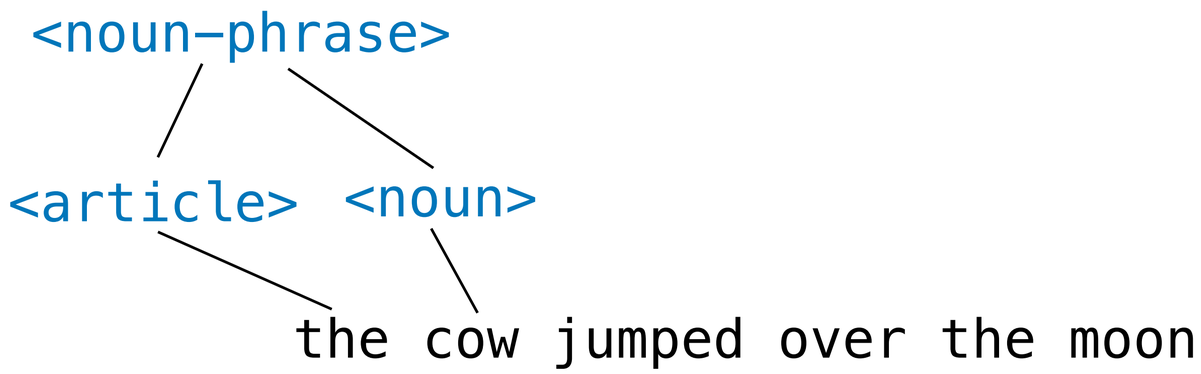
A Derivation

<sentence>
<noun-phrase> <verb-phrase>
<noun-phrase> <verb> <prep-phrase>
<noun-phrase> jumped <prep-phrase>
<noun-phrase> jumped <prep> <noun-phrase>
<noun-phrase> jumped over <noun-phrase>
<noun-phrase> jumped over <article> <noun>
<noun-phrase> jumped over <article> moon
<noun-phrase> jumped over the moon



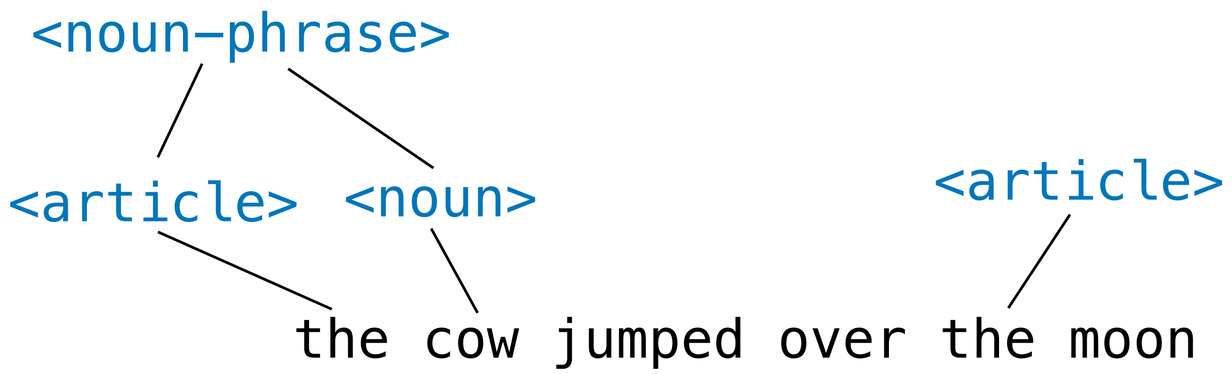
A Derivation

<sentence>
<noun-phrase> <verb-phrase>
<noun-phrase> <verb> <prep-phrase>
<noun-phrase> jumped <prep-phrase>
<noun-phrase> jumped <prep> <noun-phrase>
<noun-phrase> jumped over <noun-phrase>
<noun-phrase> jumped over <article> <noun>
<noun-phrase> jumped over <article> moon



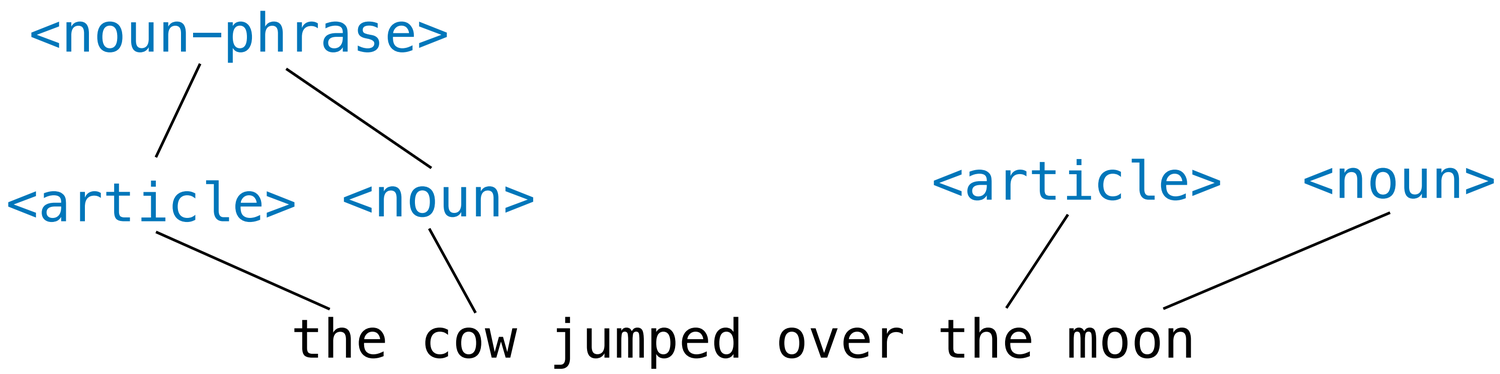
A Derivation

<sentence>
<noun-phrase> <verb-phrase>
<noun-phrase> <verb> <prep-phrase>
<noun-phrase> jumped <prep-phrase>
<noun-phrase> jumped <prep> <noun-phrase>
<noun-phrase> jumped over <noun-phrase>
<noun-phrase> jumped over <article> <noun>



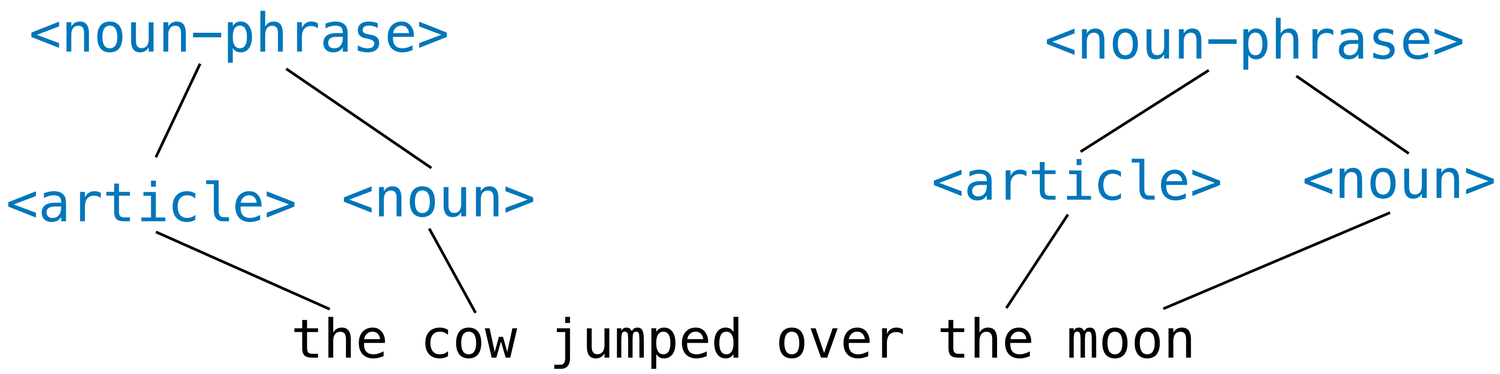
A Derivation

<sentence>
<noun-phrase> <verb-phrase>
<noun-phrase> <verb> <prep-phrase>
<noun-phrase> jumped <prep-phrase>
<noun-phrase> jumped <prep> <noun-phrase>
<noun-phrase> jumped over <noun-phrase>



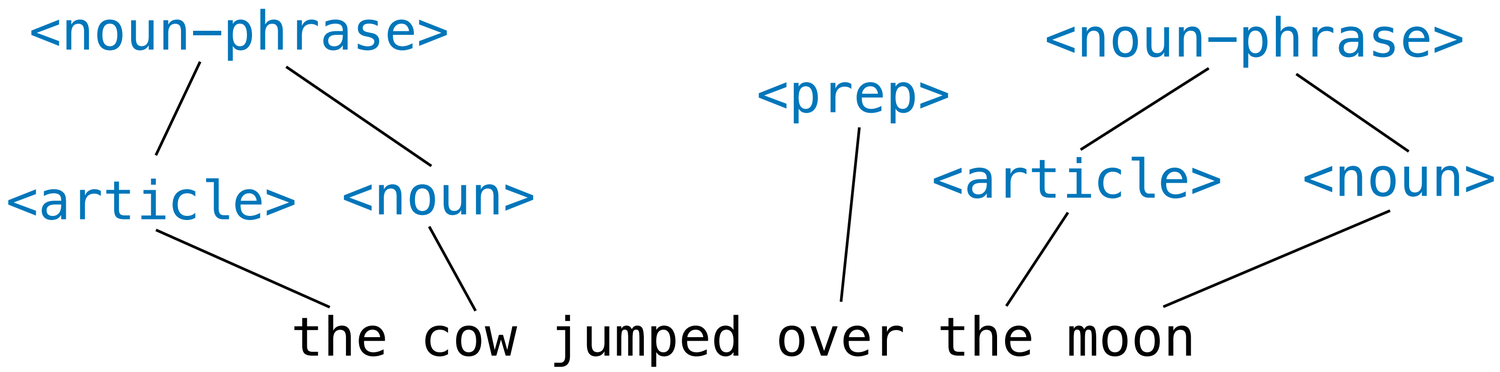
A Derivation

<sentence>
<noun-phrase> <verb-phrase>
<noun-phrase> <verb> <prep-phrase>
<noun-phrase> jumped <prep-phrase>
<noun-phrase> jumped <prep> <noun-phrase>



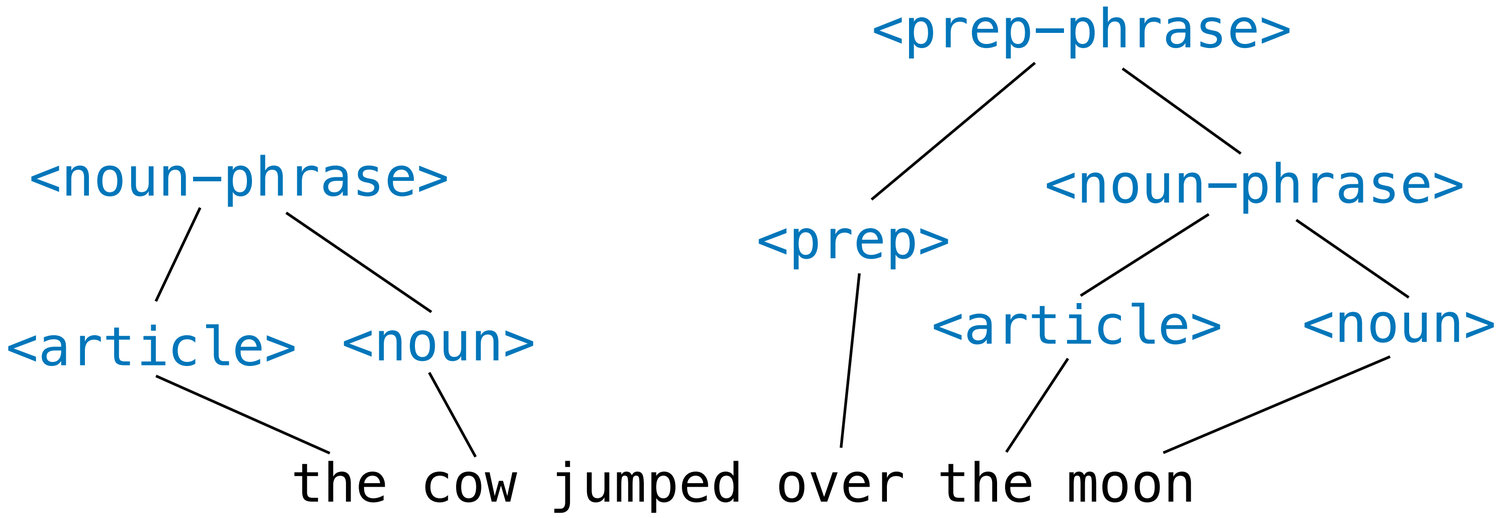
A Derivation

<sentence>
<noun-phrase> <verb-phrase>
<noun-phrase> <verb> <prep-phrase>
<noun-phrase> jumped <prep-phrase>



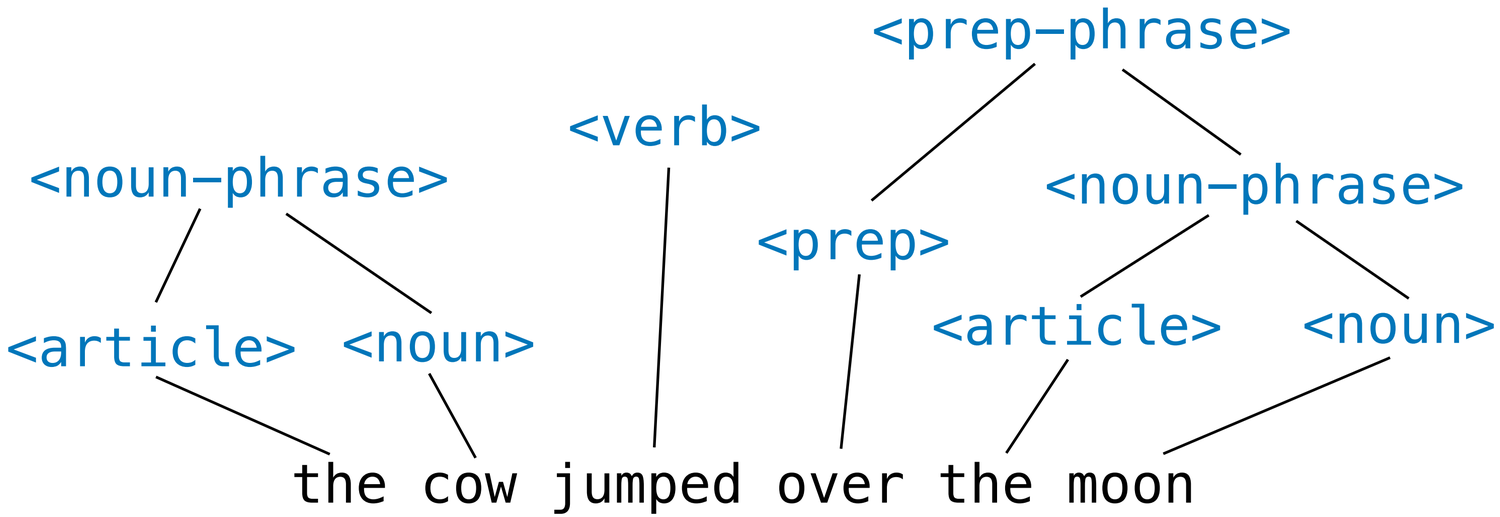
A Derivation

<sentence>
<noun-phrase> <verb-phrase>
<noun-phrase> <verb> <prep-phrase>



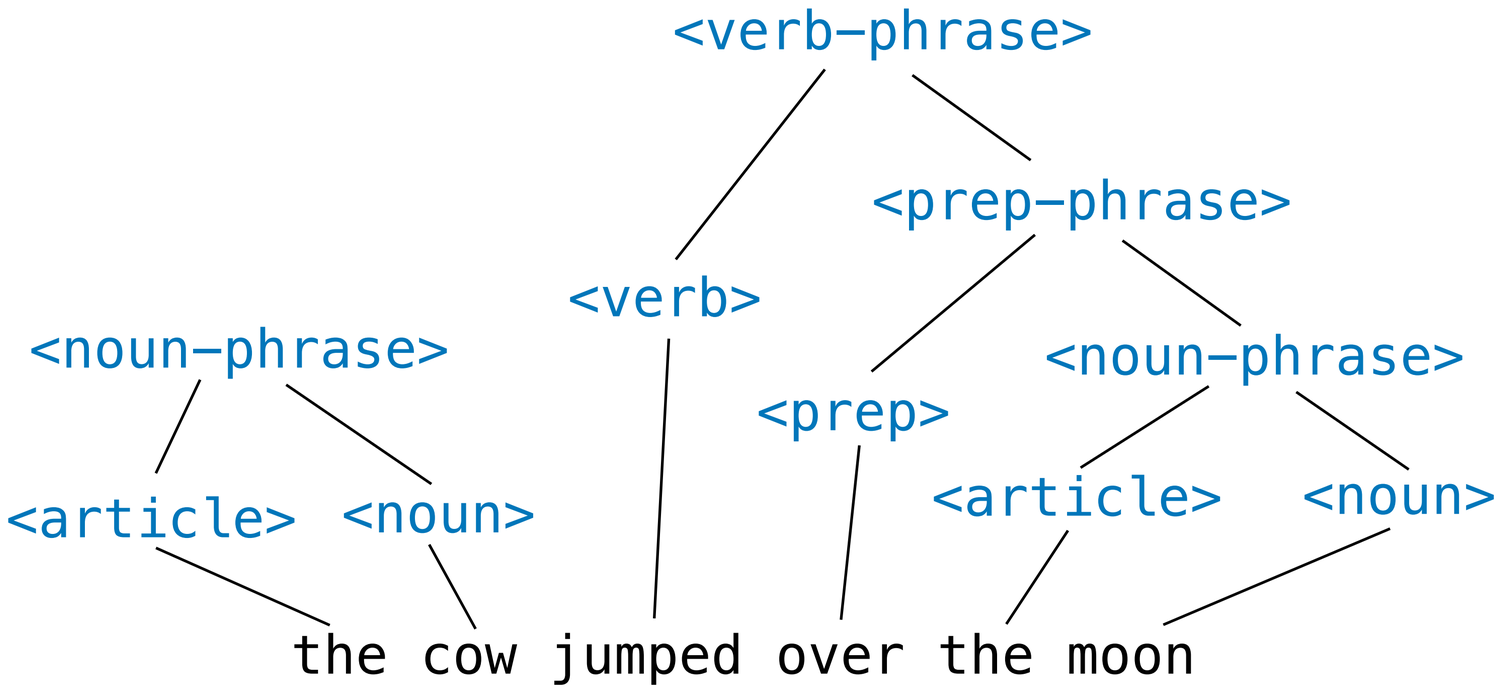
A Derivation

<sentence>
<noun-phrase> <verb-phrase>

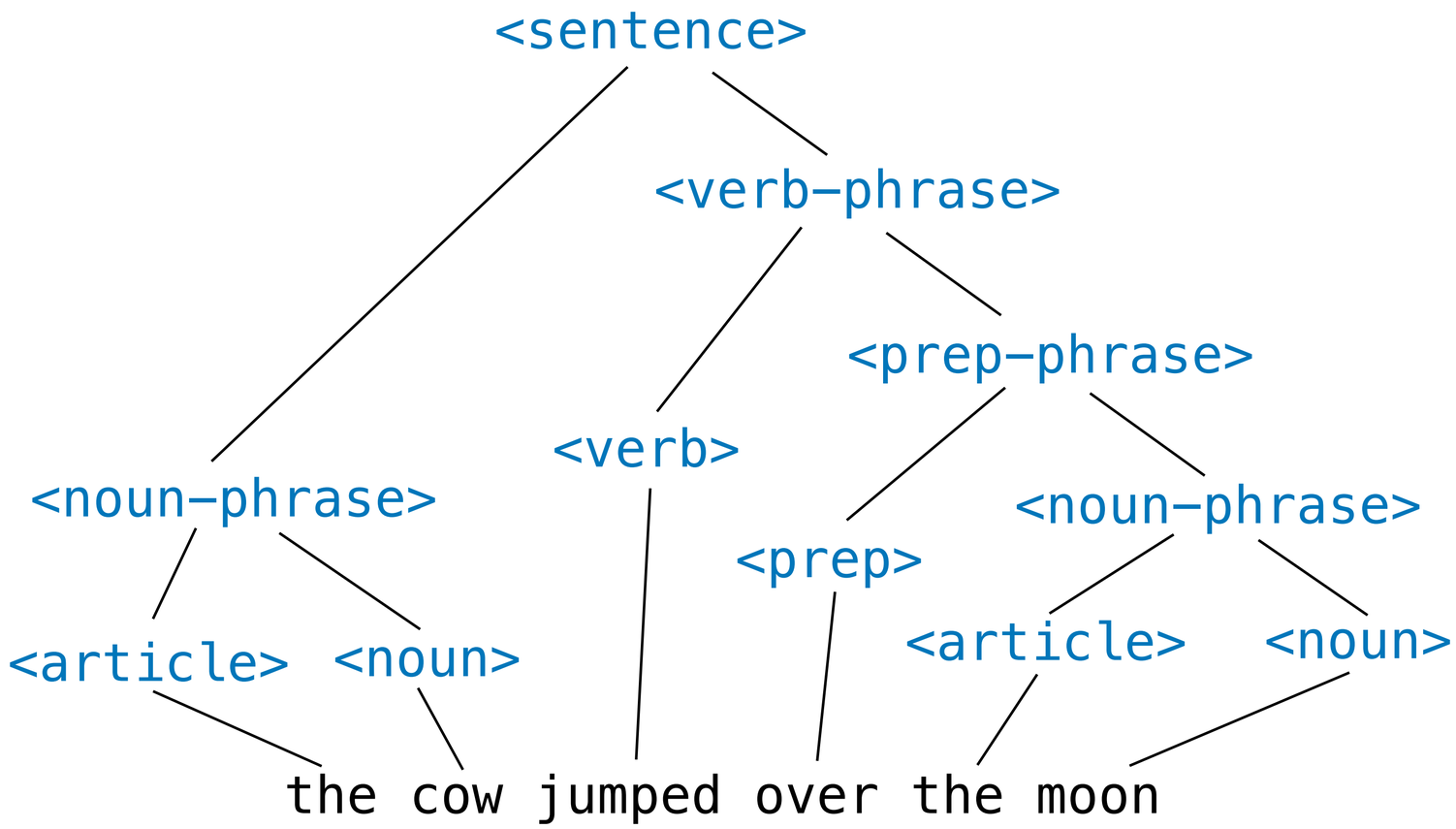


A Derivation

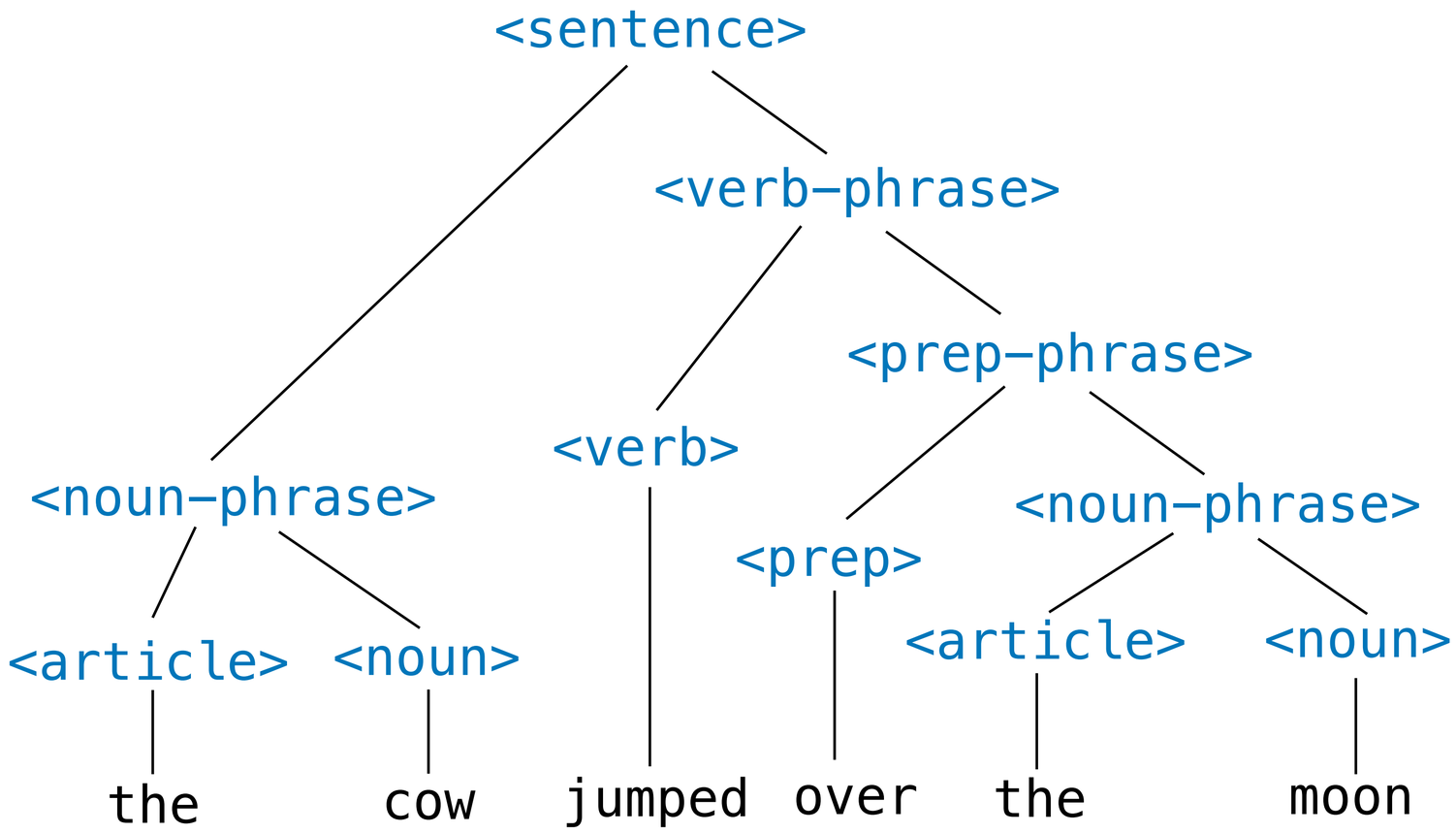
<sentence>



A Derivation



A Parse Tree



A derivation encodes hierarchical structure

Definitions (Symbols and Sentences)

<noun-phrase> jumped over <noun-phrase>

A **grammar** is defined in terms of a collection of symbols

Nonterminal symbols are symbols we will be allowed to "expand" (e.g., **<article>**)

Terminal symbols are symbols cannot be further expanded (e.g. *moon*)

A **sentential form** is a sequence of terminal or nonterminal symbols

A **sentence** is a sequence of *only terminal* symbols

Production Rules

```
<non-term> ::= sent-form1 | sent-form2 | ...
```

A **(BNF) production rule** describes what we can replace a non-terminal symbol with in a derivation

The "|" means: we can replace it with one **or** the other sentential forms on either side of the "|"

Recall: Let-Expressions (Syntax Rule)

<expr> ::= let <var> = <expr> in <expr>

If x is a valid variable name, and e_1 is a well-formed expression and e_2 is a well-formed expression then

let x = e_1 in e_2

is a well-formed expression

Recall: Let-Expressions (Syntax Rule)

```
<expr> ::= let <var> = <expr> in <expr>
```

BNF Grammar

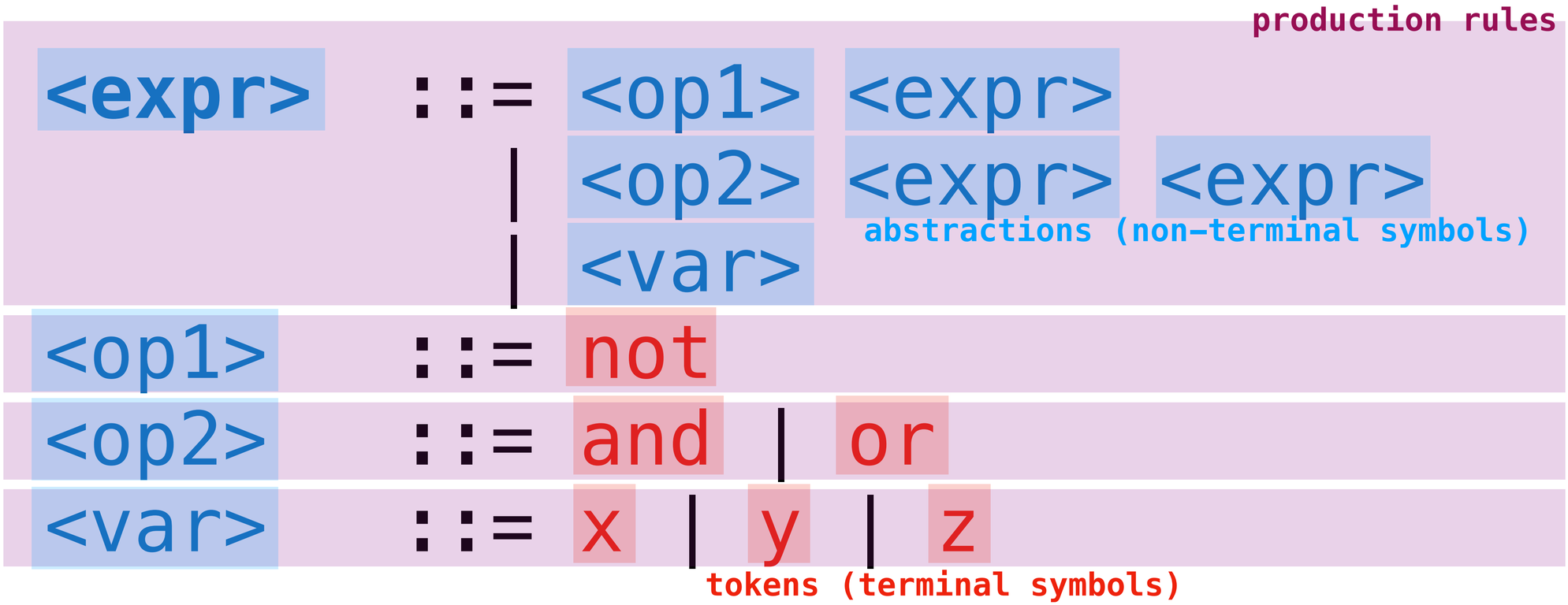
A **BNF grammar** is defined by a collection of production rules and a **starting (nonterminal) symbol**

Note. We don't specify the symbols of a grammar, they are implicit in the rules

Note. We don't specify the start symbol, it's the left nonterminal symbol in the **first rule**

```
<sentence>    ::= <noun-phrase> <verb-phrase>
<verb-phrase> ::= <verb> <prep-phrase>
                | <verb>
<prep-phrase> ::= <prep> <noun-phrase>
<noun-phrase> ::= <article> <noun>
<article>     ::= the
<noun>        ::= cow
                | moon
<verb>        ::= jumped
<prep>        ::= over
```

Another Example



Derivations and Parse Trees

Definition. A **derivation** is a sequence of sentential forms (beginning at the start symbol) in which each form is the result of **replacing one non-terminal symbol in the previous form** according to a production rule

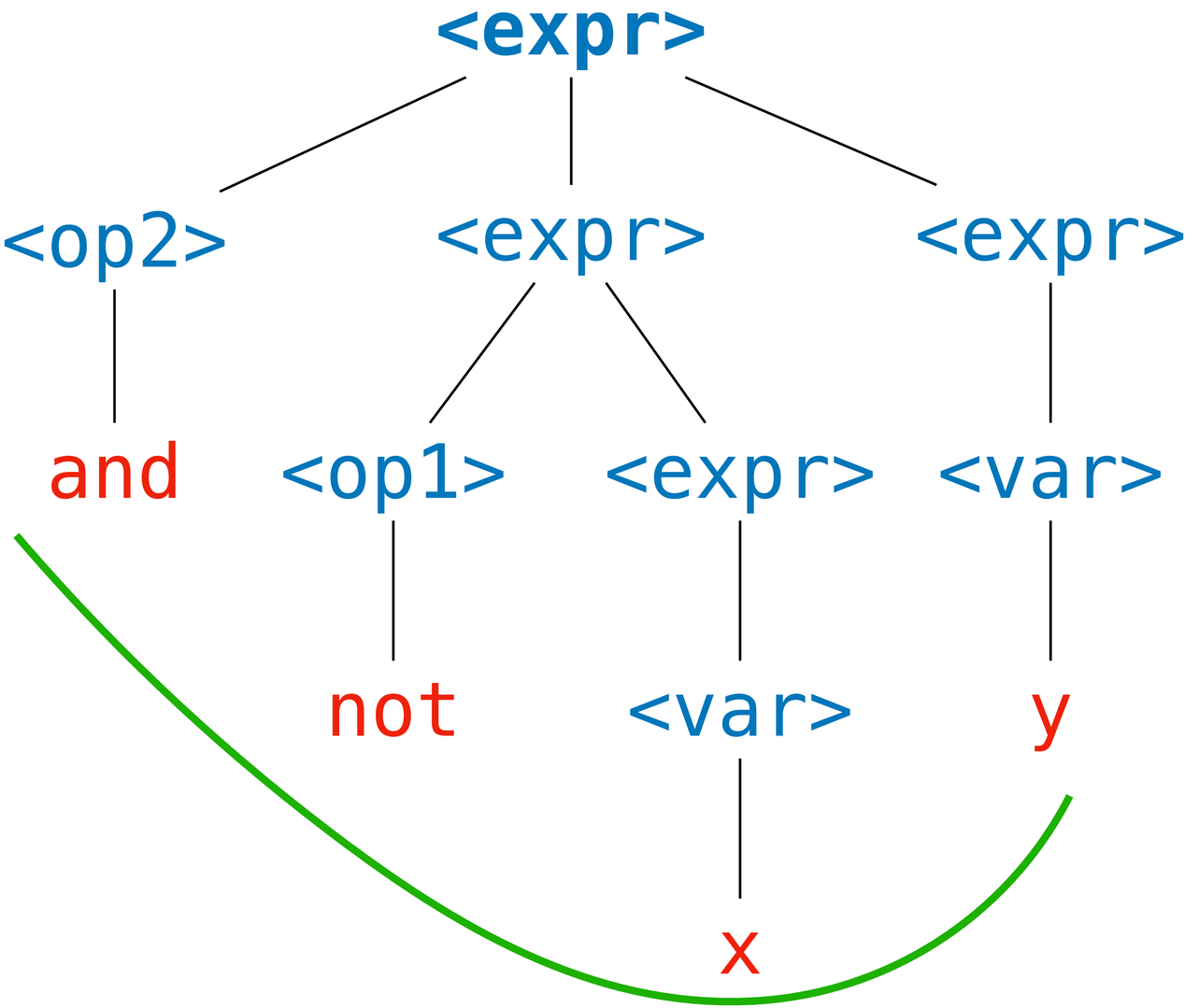
Definition. A **leftmost derivation** is a derivation in which the leftmost nonterminal symbol is replaced in each line

```
<expr>
<op2> <expr> <expr>
and <expr> <expr>
and <op1> <expr> <expr>
and not <expr> <expr>
and not <var> <expr>
and not x <expr>
and not x <var>
and not x y
```

Derivations and Parse Trees

```
<expr>  ::= <op1> <expr>
          | <op2> <expr> <expr>
          | <var>
<op1>   ::= not
<op2>   ::= and | or
<var>   ::= x | y | z
```

```
<expr>
<op2> <expr> <expr>
and <expr> <expr>
and <op1> <expr> <expr>
and not <expr> <expr>
and not <var> <expr>
and not x <expr>
and not x <var>
and not x y
```



The point: parse trees and derivations represent the same hierarchical structure

Why do we care?



We will parse **token streams** into **parse trees**

It is much easier to evaluate something hierarchical than something which is linear

Practice Problem

```
<expr>  ::= <op1> <expr>
          | <expr> <op2> <expr>
          | <var>
<op1>   ::= not
<op2>   ::= and | or
<var>   ::= x | y | z
```

*Give a derivation of **not x and y or z** in the above grammar, both as a sequence of sentential forms and as a parse tree*

Answer

<expr>

<expr> <op2> <expr>

<op1> <expr> <op2> <expr>

not <expr> <op2> <expr>

not <var> <op2> <expr>

not x <op2> <expr>

not x and <expr>

not x and <expr> <op2> <expr>

not x and <var> <op2> <expr>

not x and y <op2> <expr>

```
<expr>  ::= <op1> <expr>
          | <expr> <op2> <expr>
          | <var>
<op1>   ::= not
<op2>   ::= and | or
<var>   ::= x | y | z
```

not x and y or z

not x and y or <expr>

not x and y or <var>

not x and y or z

Answer

```
<expr>  ::= <op1> <expr>
          | <expr> <op2> <expr>
          | <var>
<op1>   ::= not
<op2>   ::= and | or
<var>   ::= x | y | z
```

not x and y or z

<expr>

<op1> <expr>

not <expr>

not <expr> <op2> <expr>

not <var> <op2> <expr>

not x <op2> <expr>

not x and <expr>

not x and <expr> <op2> <expr>

not x and <var> <op2> <expr>

not x and y <op2> <expr>

not x and y or <expr>

not x and y or <var>

not (x and (y or z))

An Example from 320Caml

```
let x = 2 in if x = z then x else y
```

How can we demonstrate that this is a well-formed expression?

Answer: Well build a derivation/parse tree for it with the root **<expr>**!

Recall: Let-Expressions (Syntax Rule)

<expr> ::= let <var> = <expr> in <expr>

If x is a valid variable name, and e_1 is a well-formed expression and e_2 is a well-formed expression then

let x = e_1 in e_2

is a well-formed expression

Recall: If-Expressions (Syntax Rule)

<expr> ::= if <expr> then <expr> else <expr>

If e_1 is a well-formed expression and e_2 is a well-formed expression and e_3 is a well-formed expression, then

if e_1 then e_2 else e_3

is a well-formed expression

let x = 2 in if x = z then x else y

<expr>

let <var> = <expr> in <expr>

let x = <expr> in <expr>

let x = <intlit> in <expr>

let x = 2 in <expr>

let x = 2 in if <expr> then <expr> else <expr>

let x = 2 in if <expr> <op2> <expr> then <expr> else <expr>

let x = 2 in if <var> <op2> <expr> then <expr> else <expr>

let x = 2 in if x <op2> <expr> then <expr> else <expr>

let x = 2 in if x = <expr> then <expr> else <expr>

let x = 2 in if x = <var> then <expr> else <expr>

```
let x = 2 in if x = z then x else y
```

let x = 2 in if x = z then <expr> else <expr>

let x = 2 in if x = z then <var> else <expr>

let x = 2 in if x = z then x else <expr>

let x = 2 in if x = z then x else <var>

let x = 2 in if x = z then x else y

Ambiguity

Ambiguity in Natural Language

The duck is ready for dinner.

John saw the man on the mountain with a telescope.

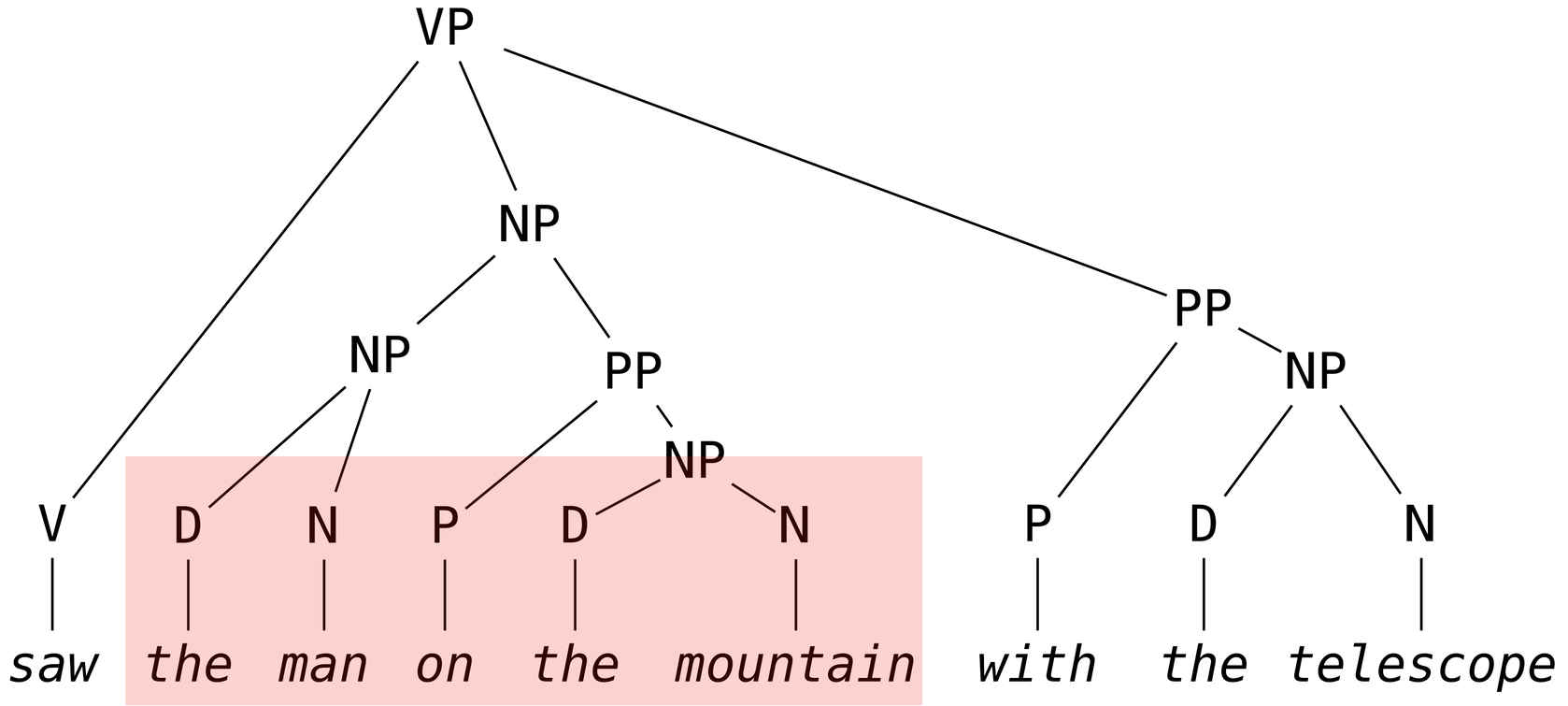
He said on Tuesday there would be an exam.

Natural language has ambiguities that can confuse the meaning of a sentence

We have informal tactics for avoiding these pitfalls

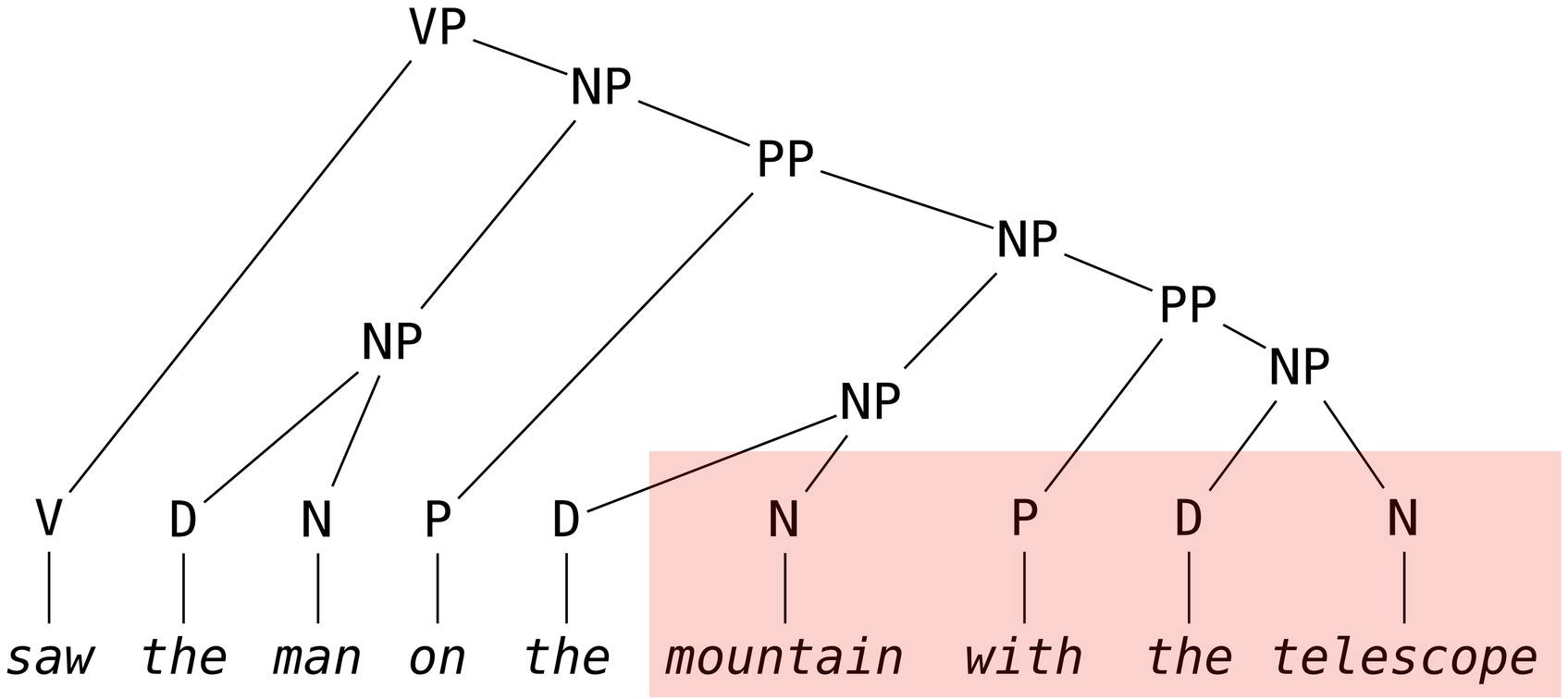
(examples taken from around the internet)

Aside: Ambiguity and Linearity



Ambiguity is caused by writing down hierarchical structures in a linear fashion

Aside: Ambiguity and Linearity



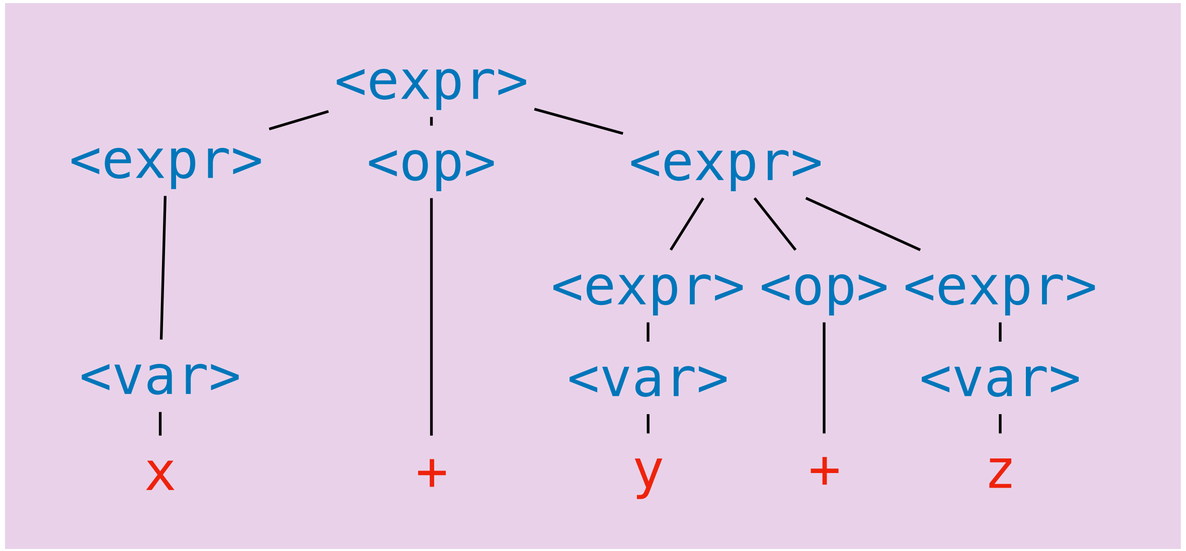
If we know the parse tree, then there is no ambiguity

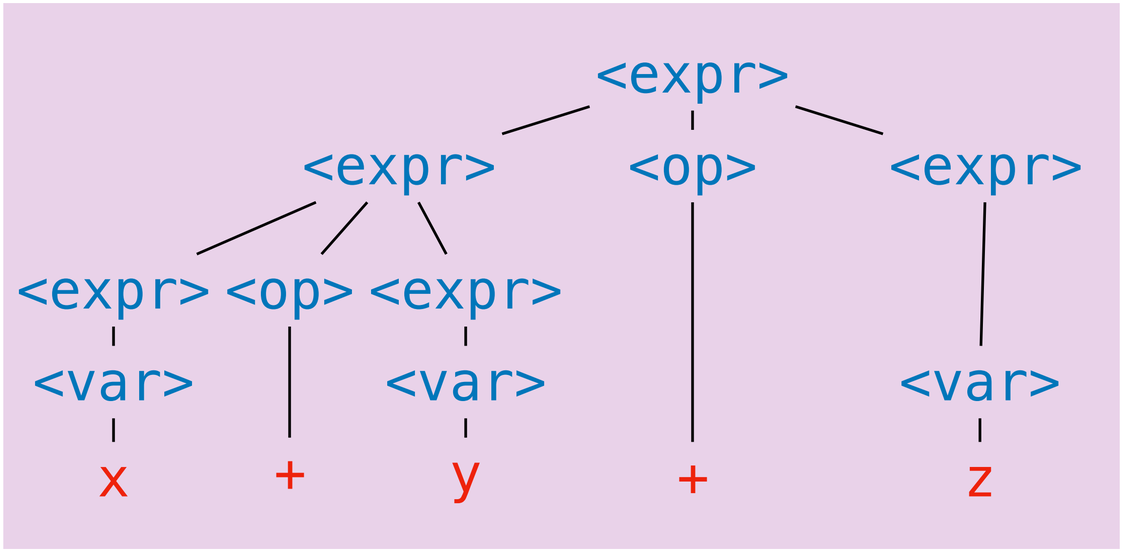
Ambiguity in Formal Grammar

Definition. A BNF grammar is **ambiguous** if there is a sentence with multiple parse trees/ leftmost derivations

```
<expr> ::= <expr> <op> <expr>
         | <var>
<op>   ::= +
<var>  ::= x | y | z
```

x + y + z can be derived as





Example

```
<expr> ::= x
         | if <expr> then <expr>
         | if <expr> then <expr> else <expr>
```

if x then (if x then x else x)

if x then (if x then x) else x

What can we do about ambiguity?

Summary

When we specify a PL (e.g., in the final project), you will be given a **BNF grammar**

You may need to translate this into a parser

So you will need *practice reading BNF specifications*

Ambiguity means two distinct parse trees for the same expression; we'll fix it next lecture